

Organized swarms on Hanabi

What the company does in Sandbox 1, and the words we use for it

Enrico Yao-Bate October 8, 2026

1 What the company is doing

Mill Street Labs builds long-running populations of agents that discover useful knowledge, share it, and build on it. The object of study is the population’s organization, not its agents: who teaches whom, how a learner checks what it is taught, how credit and compute flow, how groups compete.

The thesis is that such populations can improve collectively through two mechanisms, modelled on how human innovation builds on knowledge accumulated over generations. The first is *adaptive social dynamics*: groups compete for finite resources, so helping one’s group is rewarded even when it costs the individual, and agents revise how they cooperate and migrate between groups. The second is *intelligence transfer*: agents are credited for the improvement of those they teach, teaching combines discoveries from separate lineages, and the population accrues a corpus of verified knowledge.

A natural question is why not test this where it pays. The issue is that “organization A improves a population faster than B ” compares two noisy trajectories and is only as good as the evaluator that scores them. A *sandbox* is a domain whose evaluator is cheap, exact and absolute, where the counterfactual “what if this message had not been delivered” can be replayed, and where fixed reference points exist. Sandbox 1 is strategy development in skillful games; this document is about Hanabi, the game on this branch (the main branch runs a predator-prey ecology and the Flag Game). Sandbox 2 is autonomous research in financial markets, market making on Kalshi, where the evaluator is noisy, nonstationary and expensive to replay.

The two are tied by one constraint: everything in Section 3 must carry over to Sandbox 2 with only the evaluator swapped, games of Hanabi here and a strategy’s profit there. A finding that depends on Hanabi rather than on the organization does not count. The dials of Section 4 and the measurements of Section 5 are used by name in every design note and finding.

2 Hanabi

Hanabi is a cooperative card game; we play it with two players, one team against the deck. The deck has 50 cards in five colours: three 1s, two each of 2, 3, 4, and one 5 per colour. The team builds one stack per colour in the order $1, \dots, 5$, and the score is the number of cards on the stacks, at most 25. Each player holds five cards facing away, seeing the partner’s hand and never their own. The table holds eight clue tokens and three lives. A turn is one of three moves. A *clue* spends a token and names one colour or one rank, *touching* every matching card in the partner’s hand. A *discard* throws away one’s own card, regains a token, and draws. A *play* puts one of one’s cards on its stack: if it is the next rank it scores, otherwise it is a *bomb* and costs a life. The game ends when the stacks are complete, when the third life is lost (our engine then scores 0), or one round after the deck runs out.

A count shows why clues are the heart of the game. After the deal 40 cards remain, and each play or discard draws one, so a game that plays all 25 cards discards at most 17 (two plays can come after the deck is empty). A token comes back only with a discard or a finished stack, so the team can give at most $8 + 17 + 5 = 30$ clues in the whole game, about one per card it must play. Literally, a clue states one attribute of the cards it touches, and knowing that a card is safe to play takes in general two, its colour and its rank. A team that uses clues only for their literal content therefore cannot reach 25; good teams agree in advance on rules that make a clue mean more than it says.

Definition 2.1 (Convention). A *convention* is a rule, held by both players, that maps a clue in its context to a meaning beyond its literal content.

Example 2.2 (One clue, two meanings). The red stack is empty and we clue “red”, touching a single card. If the card is the red 1 and we mean “play it”, a partner holding “a clue on one new card means play it” plays it blind and scores, while a partner holding “a colour clue means keep this card” holds it, perhaps all game. If the card is the red 5, the only one in the deck, and we mean “keep it”, the second partner keeps it and the first plays it, bombs, and caps the red stack at 4 forever. Neither convention is better alone; the outcome of the clue depends only on whether the rule is shared.

This is what makes Hanabi a laboratory for culture: a convention is worth something only when the partner holds it too, so it must be transmitted to have value, and whether two players share one is measurable by letting them play.

Definition 2.3 (Deal, bot, self-play, cross-play). A *deal* is one shuffled deck, indexed by a seed. A *bot* is a deterministic program π from what one player sees (the partner’s cards, stacks, discards, tokens, clues received, last moves) to a legal move, reset before every game. For bots π, π' and a finite set of deals $\mathcal{D}$, with seats alternating across deals,

$$\hat{V}_{\mathcal{D}}(\pi, \pi') := \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{score}(\pi, \pi'; d).$$

$\hat{V}_{\mathcal{D}}(\pi, \pi)$ is the *self-play* score of π ; $\hat{V}_{\mathcal{D}}(\pi, \pi')$ for $\pi \neq \pi'$ is their *cross-play* score.

Remark 2.4 (Why cross-play measures shared culture). Self-play rewards internal consistency and says nothing about compatibility. Hu et al. (*Other-Play*, 2020) report reinforcement-learning agents that each score about 24 with themselves and about 2.5 with an independently trained partner: each converged on private signalling rules the other reads differently, which is Example 2.2 at scale. This collapse between independent lineages is the null against which every claim of transfer is measured.

Definition 2.5 (Reference bots). A *reference bot* (the code says *anchor*) is a fixed bot with a known score. Ours are three rule bots from Canaan et al. (AIIDE 2020), each an ordered list of rules in which the first that applies decides the move. Over 1,000 deals in our engine: *Piers* 16.99, the published number reproduced move for move; *IGGI* 15.86 (published 15.99); *Flawed* 0, a deliberately bad list. Findings call them the 17-, 16- and 0-point bots.

Piers, for instance, plays a card known to be playable; else a card at least 60% likely to be playable if it can spare a life; else clues a partner’s playable card; else discards. *Flawed* is a similar list that plays any card at least 25% likely to be playable and clues at random; it bombs out. *Piers* is our known-good teacher, *Flawed* our known-bad one. Human players on `hanab.live` maintain a written rulebook of conventions, organized in levels and revised by pull request, and expert pairs following it score about 23 to 24.

A game is fast: a few milliseconds on one core, about a millisecond of wall clock with the development machine’s twelve workers (1,400 games per second for *Piers*). A revision by our local model takes about a minute, the wall clock of order 10^5 games. Evaluation is nearly free; model calls are the currency the population spends.

3 How a swarm works on it

Picture a population of programmers, each maintaining their own Hanabi bot, who can send one another their code and notes; the game is only the test bench. The *harness* is our code around it: it runs games, verifies, signs evidence, enforces budgets, and records every model call.

Definition 3.1 (Artifact, agent, incumbent). An *artifact* is a bot’s code (at most 400 lines of Python, run in a sandbox that cannot see its own cards or the file system) together with a *conventions document*, a plain-language statement of the rules the code implements. An *agent* is a process driven by a language model that owns exactly one current artifact, its *incumbent*.

The conventions document exists because code is a poor carrier of an idea: a reader must reverse-engineer which rule does the work, while “a clue on one new card means play it” says it.

Definition 3.2 (Revision, selection). A *revision* is one model call whose prompt holds the incumbent’s code and conventions, a summary of its scores, the traces of its three lowest-scoring games on this generation’s feedback deals, what it received this generation, and up to three artifacts from its group’s corpus. It returns new code and conventions, a *candidate*; a *selection* rule decides whether the candidate replaces the incumbent (default: if it scores at least as well on the same deals).

Definition 3.3 (Message, evidence). A *message* from a *teacher* to a *student* carries an artifact, a note the teacher writes about it (one model call), and *evidence*: the artifact’s self-play and cross-play with the reference bots, measured and signed by the harness. An agent cannot fabricate evidence, only earn it; it can write a persuasive note about a bad bot.

Definition 3.4 (Verify, adopt, merge, reject). To *verify* a received artifact, the received bot and the incumbent each play the same verification deals with copies of themselves; the received bot passes if its mean is higher and at most 1% of its moves are illegal. The student then *adopts* it (it becomes the incumbent), *merges* it (one model call combining the two), or *rejects* it.

Verification decides what code the student runs, not what it reads: a rejected message’s note and evidence can still sit in the next revision prompt. What a learner may read is a separate dial, the reaction rule (Section 4), and Section 6 shows the distinction matters.

Definition 3.5 (Corpus, touch log, lineage). A group’s *corpus* is a bounded store (twenty artifacts) of artifacts with evidence deposited by its members; a deposit without harness evidence is refused. Each revision retrieves up to three, and every retrieval is written to the *touch log* as (reader, artifact, generation). Every artifact records its *parents*, the incumbent it was revised from and anything adopted or merged into it; following parents gives the *lineage graph*. Lineage says what a good bot descends from and the touch log what its authors read: together, whose idea it was.

Construction 3.6 (Generation). A *generation* is one round in which every agent, in order: (1) *receives* the messages routed and delivered to it; (2) *verifies* each, and adopts, merges or rejects it; (3) *revises* its incumbent and applies the selection rule; (4) *evaluates* the incumbent: self-play on the evaluation deals, cross-play with its group and the reference bots; (5) *teaches*: sends the incumbent, a note and its evidence to the agents the routing rule names. Around these steps the population keeps its books: before them the budget is divided among groups; after evaluation, credit is assigned and artifacts are deposited; after teaching, agents migrate.

Definition 3.7 (Group, group score, compatibility, migration). A *group* is a set of agents whose messages are routed mostly among themselves and who share a corpus. Its *group score* is the expected score of two distinct members drawn at random, the *within-group cross-play*

$$\text{GS}(g) := \frac{1}{n_g(n_g - 1)} \sum_{i \neq j \in g} \hat{V}(\pi_i, \pi_j).$$

The *compatibility* of groups g, h is the *between-group cross-play* $\hat{V}(\pi_g^*, \pi_h^*)$ of their best bots. *Migration* moves an agent, with its incumbent, to another group.

This makes culture operational: individually strong but mutually incompatible bots give a low group score. Groups that never talk drift to different conventions and their compatibility falls toward the collapse of Section 2; teaching across groups and migration keep them compatible, and the lineage graph records which did the work.

Definition 3.8 (Budget). Each group has a *budget* of model calls per generation. Every call (revise, teach, merge) debits it; games do not. An allocation rule divides a fixed total across groups by their scores or credit, which is how groups compete.

Remark 3.9 (Two levels that never mix). Inside a game, two bots play about sixty turns alone, every move decided by their own code in milliseconds, with no model and no other agent involved. The population never plays a game and decides nothing by vote: each agent decides what to do with its own bot, and agents interact only through messages, verification, adoption, the corpus, the budget and group membership. The game gives an artifact its score; the population changes artifacts.

Remark 3.10 (The right analogy). The closest established frame is genetic programming: artifacts are individuals, the engine is the fitness function, the language model is the mutation operator (merging is recombination), verification and adoption are selection, and teaching is horizontal transfer, a trait passing sideways, not parent to child. What we vary is not the operator but the organization around it; a stronger model is a better operator, and our question is what organization adds at a fixed one.

4 The dials

An experiment fixes everything above and turns some of these dials; the names are used verbatim in design notes. Options marked *proposed* are not built.

Topology. Which groups can reach which: isolated, full, ring, or islands (isolated plus migration).

Routing. Who sends to whom: none, broadcast in the group, its best to all, or k random receivers.

Delivery. Whether a routed message arrives: always, or independently with known probability p .

Verification. None; self-play on N paired deals (default 200, pass if the mean is higher); that plus cross-play with reference bots; adopt, then revert if worse; a sequential test (engine only so far).

Adoption. A passed bot replaces the incumbent, is merged with it, or is ignored.

Credit. How a student's gain is attributed to teachers: none, equal split among adopted messages, the same propagated up the lineage with decay, or regression on randomized delivery.

Allocation. Splitting the budget across groups: uniform, proportional to credit, softmax of group score with a floor, or Nash averaging over groups' best bots compared deal by deal (so cloning a group buys no budget).

Selection. Whether a candidate replaces the incumbent, and which artifact is revised next: hill climbing (default), score-weighted parents from the group's archive, or sampling over the archive's lineage tree.

Migration. None, random swaps every m generations, or each group's best to the next group.

Environment change. Fixed rules, or a switch at a set generation to a variant (four colours; four cards and six tokens), to ask whether a population re-adapts.

Horizon and budget. Generations, and calls per group per generation. Teaching costs one call per teacher, so sharing organizations spend more per generation than isolated ones.

Reaction rule. What a revision may read of what was received: everything (default); only messages whose bot passed verification (*quarantine*); only those whose evidence beats the learner’s score (*evidence-gated*, proposed); an idea tried in a branch, shown if it wins (*idea trial*, proposed).

5 What we measure

Every comparison is paired: all conditions play the same deals, so deal luck cancels in differences. A *seed* fixes deals and model sampling; a *replicate* is a change of seed. The *stub* is a fake model that makes no calls; on it, each metric’s spread is its *null band*.

Self-play, cross-play on held-out deals, within and between groups (Section 3).

Frozen ladder. Every tenth generation each group’s best is frozen; the current best plays every earlier frozen bot and the reference bots, so progress is measured against fixed partners.

Calls-to-threshold. Cumulative model calls when population mean self-play first reaches 17 or 20 (censored if never); calls, not generations, because organizations differ in cost per generation.

Retained innovations. Artifacts that beat their group archive’s best when first evaluated and still have descendants ten generations later: discoveries that were kept and built on.

Innovation rate. $\iota(s)$, the probability that one revision of a bot at level s gains over 0.5 on held-out deals.

Copy fidelity. After one revision with a teacher’s message in view, the teacher’s self-play minus the copy’s (*copy loss*), and the fraction of copies that reach the teacher’s level.

Leak rate. a , the probability that verification passes a bot that is not better. With *fan-out* k (receivers per teacher) a bad artifact makes about ka new carriers per carrier per generation: to first order it dies out if $ka < 1$.

Breakdown point. Each delivered message is replaced with probability ε by a bad bot with a persuasive note (*sabotage*); ε^* is the smallest ε at which the group score falls below 70% of its sabotage-free value.

Per-message credit. Let a student receive k messages, and let $Y(S)$ be its gain in self-play on the evaluation deals when the delivered set is $S \subseteq [k]$. The *credit* of message j is the effect of delivering it, averaged over the delivery of the others,

$$\tau_j := \frac{1}{2^{k-1}} \sum_{S \subseteq [k] \setminus \{j\}} [Y(S \cup \{j\}) - Y(S)];$$

whatever the student does with j (adopt, reject, or merely read) is inside the effect. Our local model (Qwen3.6-35B-A3B, fixed seed, temperature 0) returns identical text for an identical prompt, so the update is a deterministic function of S . Every one of the 2^k subsets can therefore be replayed, and the counterfactual is observed rather than estimated; this *exact replay* is the truth. It costs 2^k revisions per student, which a population cannot afford, so the question is which cheap estimator from ordinary logs (leave-one-out, regression on randomized delivery) agrees with it.

Three deal sets. Every generation draws three disjoint sets of deals: *feedback* deals, whose worst traces the model sees; *verification* deals, on which received bots are judged; and *evaluation* deals, on which every reported score is computed and which the model never sees. The reason is selection on noise. A revision can fit the feedback deals (a rule that rescues one unlucky hand), and scoring

it there would count the fit as skill. Verification keeps the better of two bots on its deals, so the kept bot's score there is biased up, for two equally good bots by $\sigma_{\Delta}/\sqrt{2\pi N}$ in expectation (σ_{Δ} the per-deal standard deviation of their score difference, N the number of deals). Small per decision, the bias compounds over generations of selection; deals that played no part in any choice remove it. Deals are independent draws, so a random split suffices.

6 What we know so far

All on the local model unless marked engine-only or stub. A finding is *draft* until Enrico has read it, *preliminary* at pilot size or on one model, *solid* when powered or unambiguous, *overturned* when later contradicted. Details in `findings/`; one line each in `RESULTS.md`.

- [solid, this model] The local model's ceiling is about 17: it improved the 17-point bot in 0 of 30 revisions, the 16-point bot in 1 of 6, a 3-point bot in 2 of 3.
- [solid, engine-only] Verification leaks: the default rule passes $a = 6\%$ of bots that are not better; a sequential test matches a 400-deal check with 73% fewer games.
- [preliminary] Verification gates code, not ideas: a message whose bot is rejected still lowers the student by about 2.4 points through its prose, against a re-sent copy of the student's own bot.
- [preliminary] Quarantine removes it by construction; at temperature 0 nothing is left to test.
- [preliminary] Only exact replay recovers who helped: with three messages per student, true credit varies across seeds by about 2 points, a floor for any estimator that pools; pooled regression recovers the seed-averaged credit.
- [preliminary] Copying the 17-point bot loses 1.2 to 1.8 points from code, prose or both alike; copies are bimodal, exact clones plus failures.
- [preliminary, stub] Without verification a group collapses at about 4% sabotage; with it, flat to 50%. A test of the pipeline, not of models.
- [preliminary, stub] Population runs fit in a night; budgets differ while teaching costs calls.
- Organization versus isolation on a real model: nothing yet.

7 The questions on the board

1. *Does organization make a population accumulate faster than isolation?* The thesis itself; if the answer at matched calls is no, the rest is instrumentation.
2. *Can the swarm tell who helped?* Crediting agents for those they teach needs credit recoverable from logs; otherwise allocation by credit allocates noise.
3. *Do bad ideas spread, and what stops them?* A population that shares can be contaminated, and a persuasive wrong idea is the failure mode that grows with sharing.
4. *What transfers, and at what cost?* Fidelity and the price in calls and games decide whether sharing beats independent search, and both must survive the move to Sandbox 2.
5. *Where is the ceiling, the model or the game?* If the model cannot pass 17, accumulation runs show only the climb to 17, and an organization effect cannot be told apart from a limit of the operator.